

Document technique final

Plateforme de gestion d'un cabinet d'audioprothèse
Solution DevOps hybride sur Microsoft Azure

Équipe	Zaafir Mougammadou Zaccaria · Elyess Rjafellah · Adame Nianghane · Anis Douadi
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Client	Cabinet d'audioprothèse « AudioPro » (commanditaire)
Dépôt	github.com/Rifaazs27/projet-audioprothese
Nature	Rendu de groupe — solution technique complète

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Présentation de l'entreprise et de l'équipe

« AudioPro » est un cabinet d'audioprothèse qui accompagne ses patients tout au long de leur appareillage. Son activité quotidienne repose sur le suivi d'un ensemble d'informations sensibles : le dossier de chaque patient, les appareils auditifs qui lui sont posés et le calendrier des rendez-vous de contrôle. Jusqu'ici gérées de façon dispersée, ces données méritaient une plateforme centralisée, fiable et sécurisée, capable de garantir la confidentialité exigée par la réglementation sur les données de santé tout en restant disponible et simple d'usage pour les praticiens.

Le cabinet a confié à notre équipe la conception et la mise en production de cette plateforme, non pas comme une simple application isolée, mais comme un **système complet exploité selon les pratiques DevOps** : automatisation des déploiements, infrastructure décrite sous forme de code, supervision permanente, sécurité intégrée à chaque étape et capacité à se rétablir après incident.

Notre équipe est composée de quatre étudiants de la classe M2 DO C. Afin de couvrir l'intégralité du périmètre sans recouvrement inutile, nous avons réparti les responsabilités par grands domaines techniques, chacun étant pilote sur son sujet tout en contribuant aux revues de code et aux décisions d'architecture communes :

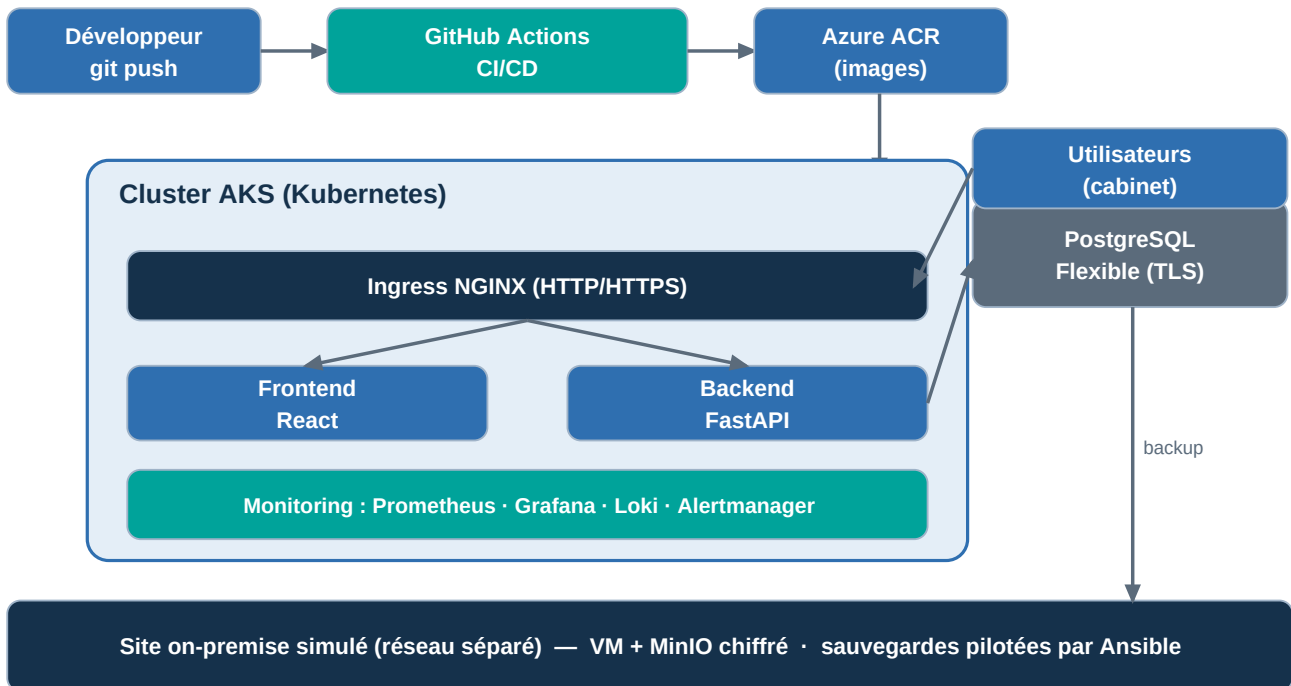
Membre	Rôle	Périmètre principal
Zaafir Mougammadou Zaccaria	Infrastructure & déploiement	Infrastructure as code Terraform (AKS, ACR, PostgreSQL, réseau, budget), application FastAPI/React, déploiement applicatif.
Elyess Rjafellah	CI/CD & automatisation	Chaîne GitHub Actions (provisionnement, build, scans, déploiement, sauvegarde), état Terraform distant, pipeline auto-réparant.
Adame Nianghane	Observabilité & sécurité	Supervision Prometheus/Grafana/Loki/Alertmanager, DevSecOps (Trivy, CodeQL, Gitleaks), RBAC, TLS, conformité RGPD.
Anis Douadi	Kubernetes & on-premise / PRA	Packaging Helm, autoscaling et politiques, site on-premise simulé, MinIO chiffré, Ansible, plan de reprise d'activité.

2. Analyse de la problématique et solution proposée

La problématique dépasse le simple développement d'un logiciel. Les données manipulées étant des données de santé, la **confidentialité** et la **traçabilité** sont impératives. Le service devant être utilisé au quotidien par le cabinet, sa **disponibilité** et sa capacité d'**auto-rétablissement** sont essentielles. La solution doit en outre rester **économe** — le projet s'inscrit dans le cadre d'un compte Azure for Students plafonné à 85 dollars — et entièrement **reproductible**.

Pour répondre à ces exigences, nous avons conçu une application web conteneurisée — une interface React, une API FastAPI et une base PostgreSQL managée — déployée sur un cluster Kubernetes managé (Azure AKS) et encadrée par une chaîne DevOps de bout en bout. Conformément au cahier des charges, l'architecture est **hybride** : le cloud héberge l'application tandis qu'un site on-premise (simulé par un réseau séparé) accueille le stockage objet chiffré des sauvegardes.

Le schéma ci-dessous synthétise cette architecture, du poste du développeur jusqu'à l'exécution supervisée en production et la réplication des sauvegardes vers l'on-premise :



Le flux est le suivant : chaque modification poussée par un développeur déclenche le pipeline d'intégration et de déploiement, qui teste le code, en vérifie la sécurité, construit les images, les publie dans Azure Container Registry puis les déploie sur le cluster via Helm. L'application est exposée aux utilisateurs par un Ingress unique ; le backend dialogue avec PostgreSQL en TLS ; la supervision et les sauvegardes fonctionnent en continu et de façon autonome.

3. Gestion des coûts — démarche FinOps (M2)

Le FinOps (Financial Operations) désigne la discipline consistant à donner à une équipe technique la responsabilité et la visibilité de ses dépenses cloud, afin de prendre des décisions d'architecture éclairées par leur coût. Dans notre projet, cette discipline n'a pas été un exercice théorique : le crédit étudiant est plafonné à **85 dollars, non rechargeable et sans carte bancaire de secours**. Chaque choix technique a donc été pesé à l'aune de son impact financier, ce qui correspond précisément à l'esprit de la spécialisation M2.

3.1 La démarche adoptée : informer, optimiser, opérer

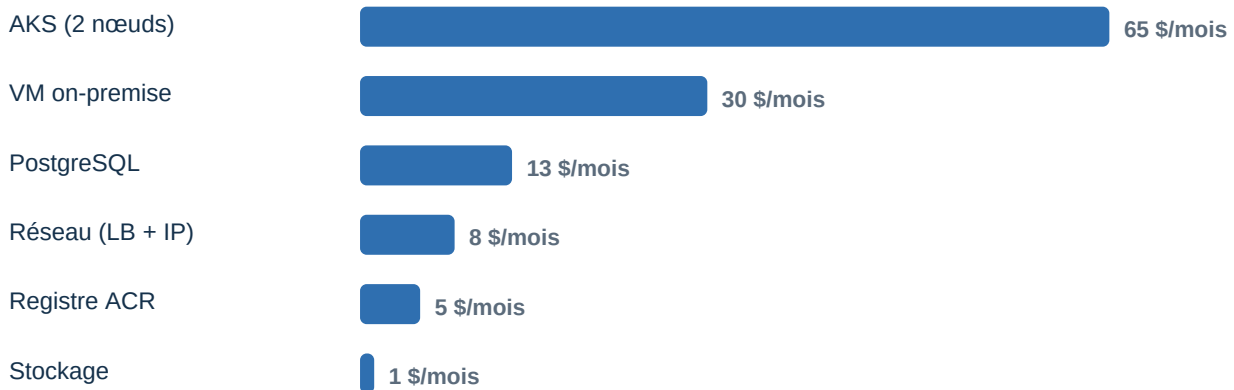
Nous avons structuré notre travail selon les trois phases du cycle FinOps de référence. La phase d'**information** a consisté à rendre chaque coût visible : étiquetage systématique des ressources, estimation préalable avant tout déploiement et suivi via Azure Cost Management. La phase d'**optimisation** a porté sur le dimensionnement au plus juste et le choix d'alternatives moins coûteuses à service équivalent. La phase d'**opération** a industrialisé la maîtrise des coûts : budget avec alertes, extinction et reconstruction automatisées, de sorte que la sobriété ne repose pas sur la discipline manuelle mais sur des mécanismes outillés.

3.2 Inventaire et estimation détaillée des coûts

Le tableau ci-dessous détaille, ressource par ressource, le dimensionnement retenu, le coût indicatif mensuel si la ressource fonctionnait en continu, et la justification du choix. Les montants sont donnés en dollars, à titre indicatif, sur la base des tarifs de la région Poland Central :

Ressource	Dimensionnement	Coût 24/7	Justification du choix
AKS — plan de contrôle	Free tier	0 \$	Kubernetes managé sans SLA payant : suffisant pour un MVP.
AKS — nœuds	2 × Standard_B2s_v2	≈ 60–70 \$	Machines « burstable » ; 2 nœuds pour héberger app + ingress + monitoring.
Base de données	PostgreSQL Flexible B1ms	≈ 13 \$	Palier burstable le moins cher, adapté à un volume modéré.
Registre d'images	ACR Basic	≈ 5 \$	SKU minimal, quota d'images largement suffisant.
VM on-premise	Standard_B2s_v2	≈ 30 \$	Héberge MinIO ; éteignable ou désactivable via une variable.
Réseau (LB + IP)	Standard	≈ 6–9 \$	Un seul point d'entrée public mutualisé par l'Ingress.
Stockage état + backups	Standard_LRS	≈ 1 \$	Redondance locale, volumétrie faible.

La répartition mensuelle de ces coûts (en fonctionnement continu) fait clairement ressortir la prédominance du calcul, sur lequel se sont concentrés nos efforts d'optimisation :



Le poste le plus lourd est le pool de nœuds AKS, suivi de la machine virtuelle on-premise. C'est logiquement sur ces deux postes que nous avons concentré nos efforts d'optimisation, en privilégiant des machines « burstable » de la série B — dont le tarif est réduit tant que la charge moyenne reste faible, ce qui est le cas d'un MVP — et en rendant la VM on-premise entièrement optionnelle grâce à une variable Terraform permettant de la désactiver lorsque le volet hybride n'est pas démontré.

3.3 Scénarios d'exploitation et projection budgétaire

Le coût réel d'un projet cloud dépend moins du dimensionnement que de la **durée pendant laquelle les ressources restent allumées**. Nous avons donc raisonné en scénarios plutôt qu'en simple montant mensuel :

Scénario d'exploitation	Principe	Coût estimé sur le semestre
Tout allumé en continu (24/7)	Infrastructure jamais arrêtée pendant 6 mois	≈ 650–750 \$ — dépasse très largement le crédit de 85 \$
Allumage à la demande (start/stop)	Cluster et VM démarrés seulement pour travailler/tester	≈ 30–50 \$ sur le semestre
Destruction après chaque session (retenu)	terraform destroy après chaque démo, reconstruction en ~14 min	≈ 10–20 \$ sur le semestre

Ce raisonnement a été déterminant : laisser l'infrastructure allumée en permanence aurait épuisé le crédit en moins d'un mois. En adoptant une logique de **destruction après chaque session** — rendue indolore par l'automatisation, puisque l'ensemble se reconstruit en une douzaine de minutes — nous avons ramené la dépense réelle à une fraction du crédit disponible, tout en conservant la capacité de démontrer une infrastructure complète à tout moment.

3.4 Leviers d'optimisation appliqués

Plusieurs leviers concrets ont été mis en œuvre. Le premier est le **rightsizing** : nous n'avons provisionné que la puissance strictement nécessaire (plan de contrôle Kubernetes gratuit, nœuds burstable, base au palier le plus bas), quitte à accepter des performances plafonnées sous forte charge, acceptables pour un MVP. Le deuxième levier est le **choix d'outils économes** : pour la centralisation des journaux, nous avons retenu Loki plutôt qu'Elasticsearch, car Loki n'indexe que les métadonnées et consomme une fraction des ressources — un point développé au paragraphe suivant.

Le troisième levier concerne la **rétenion des données** : les métriques ne sont conservées que quelques jours et les sauvegardes une semaine, ce qui limite le stockage facturé. Le quatrième est un **autoscaling borné** : le nombre de répliques de l'API peut augmenter automatiquement sous charge, mais dans une limite haute stricte, afin qu'un pic — ou une erreur — ne puisse jamais provoquer une dérive de coût. Enfin, nous avons **mutualisé l'exposition réseau** : Grafana est publié via l'Ingress existant grâce à un nom d'hôte nip.io, ce qui évite de provisionner une seconde adresse IP publique facturée.

3.5 Arbitrages structurants et alternatives écartées

Certains choix relèvent d'un arbitrage assumé entre l'exhaustivité décrite dans le cahier des charges et la sobriété imposée par le budget. Le tableau suivant récapitule ces arbitrages, l'option « complète » qui aurait été retenue sans contrainte, l'option effectivement mise en œuvre et le gain obtenu :

Besoin	Option « complète » (cahier)	Option retenue (MVP)	Gain FinOps
Logs	ELK (Elasticsearch)	Loki + Promtail	≈ –70 % de RAM et de stockage indexé
Secrets	Vault auto-hébergé	GitHub Secrets + Secret K8s	Aucun pod ni stockage à héberger
Haute dispo. base	Multi-zone (réplique)	Zone unique	≈ –50 % sur la base

Besoin	Option « complète » (cahier)	Option retenue (MVP)	Gain FinOps
Exposition Grafana	LoadBalancer dédié	Ingress mutualisé (nip.io)	–1 adresse IP publique
Traçage	Jaeger + backend dédié	Reporté (perspective)	Ressources économisées

Le cas d'ELK contre Loki est le plus illustratif : une pile Elasticsearch réclame plusieurs gigaoctets de mémoire et un stockage indexé volumineux, ce qui aurait nécessité des nœuds plus puissants et donc bien plus coûteux. Loki, en n'indexant que les étiquettes, offre les mêmes usages (centralisation, recherche, visualisation dans Grafana) pour une empreinte très inférieure. De même, un coffre de secrets auto-hébergé aurait ajouté des composants à faire tourner et à sauvegarder, alors que les secrets de la chaîne sont parfaitement gérés par le mécanisme intégré de GitHub, sans coût d'infrastructure.

3.6 Gouvernance budgétaire et suivi

La maîtrise des coûts ne repose pas seulement sur des choix de conception, mais aussi sur une **gouvernance** outillée. Un budget Azure a été défini au niveau du groupe de ressources, assorti d'alertes par courriel déclenchées à 50 %, 80 % et 100 % du plafond mensuel : l'équipe est ainsi prévenue avant toute dérive. Toutes les ressources portent des étiquettes (projet, cours, gestionnaire) qui permettent de filtrer les dépenses dans Azure Cost Management et d'attribuer chaque euro à un poste précis. Le tableau d'indicateurs ci-dessous synthétise notre pilotage financier :

Indicateur FinOps	Valeur cible / observée
Coût mensuel si allumé en continu	≈ 110–125 \$ / mois
Coût réel estimé sur le semestre	≈ 10–20 \$ (grâce à la destruction à la demande)
Part du crédit de 85 \$ consommée	≈ 15–25 %
Temps de reconstruction complète	≈ 12–14 minutes (automatisé)
Seuils d'alerte budgétaire	50 % / 80 % / 100 % (courriel)

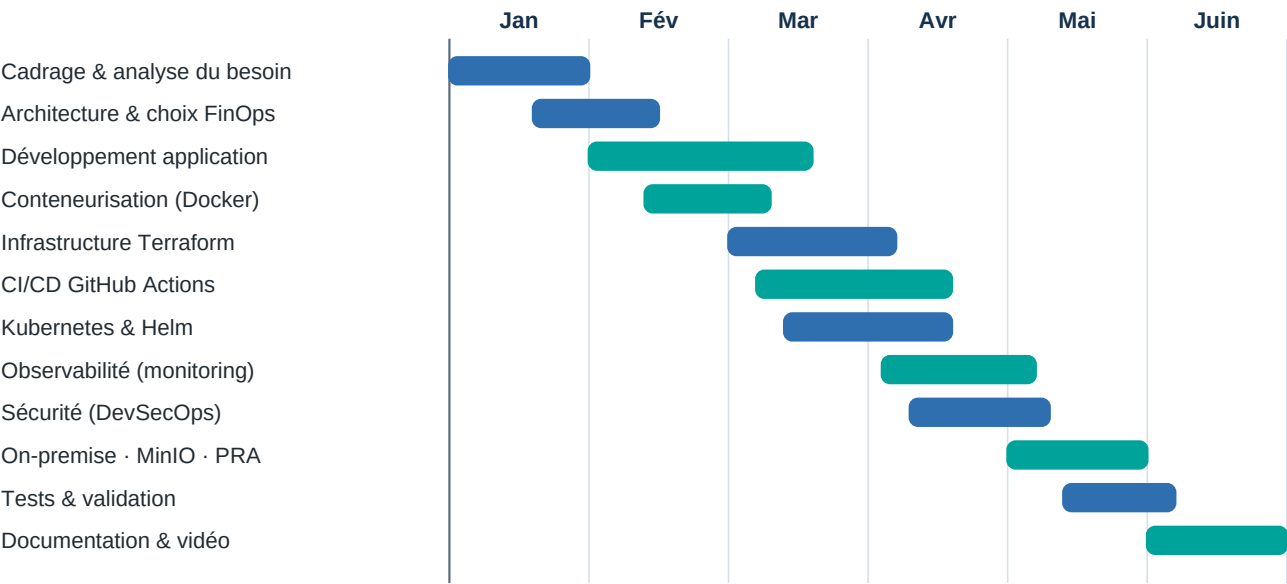
3.7 Bilan FinOps

Au terme du projet, la démarche FinOps a atteint son objectif : une infrastructure complète — cluster Kubernetes, base managée, registre, supervision, site on-premise — a été conçue, déployée et démontrée tout en ne consommant qu'une faible part du crédit de 85 dollars. Au-delà de l'économie réalisée, cette contrainte a eu une vertu pédagogique : elle nous a forcés à justifier chaque ressource, à comparer des alternatives et à intégrer le coût comme un critère de conception à part entière, au même titre que la performance ou la sécurité — ce qui constitue précisément la compétence attendue en M2.

4. Organisation de l'équipe, planification et méthodologie

Nous avons adopté une organisation agile inspirée de Scrum et de Kanban, rythmée par des sprints de deux semaines et une priorisation des tâches selon la méthode MoSCoW. Le dépôt Git a servi de point central : chaque évolution passait par une branche puis une revue de code (Pull Request) avant d'être intégrée, garantissant la qualité et le partage de connaissances entre les membres. Les outils mobilisés sont représentatifs d'une chaîne DevOps professionnelle : Git et GitHub Actions pour l'intégration et le déploiement continu, Terraform et Ansible pour la gestion de configuration, et Helm pour le packaging Kubernetes.

Le projet s'est déroulé sur le second semestre de l'année universitaire, de janvier à juin 2026. Le diagramme de Gantt ci-dessous retrace l'enchaînement des grandes phases, du cadrage initial à la rédaction des livrables et à l'enregistrement de la vidéo de démonstration :



La répartition des contributions, équilibrée entre les quatre membres, est détaillée ci-dessous. Chaque contributeur a piloté un domaine tout en participant aux décisions transverses ; la traçabilité individuelle est consultable dans l'historique du dépôt et la vue « Contributors » de GitHub.

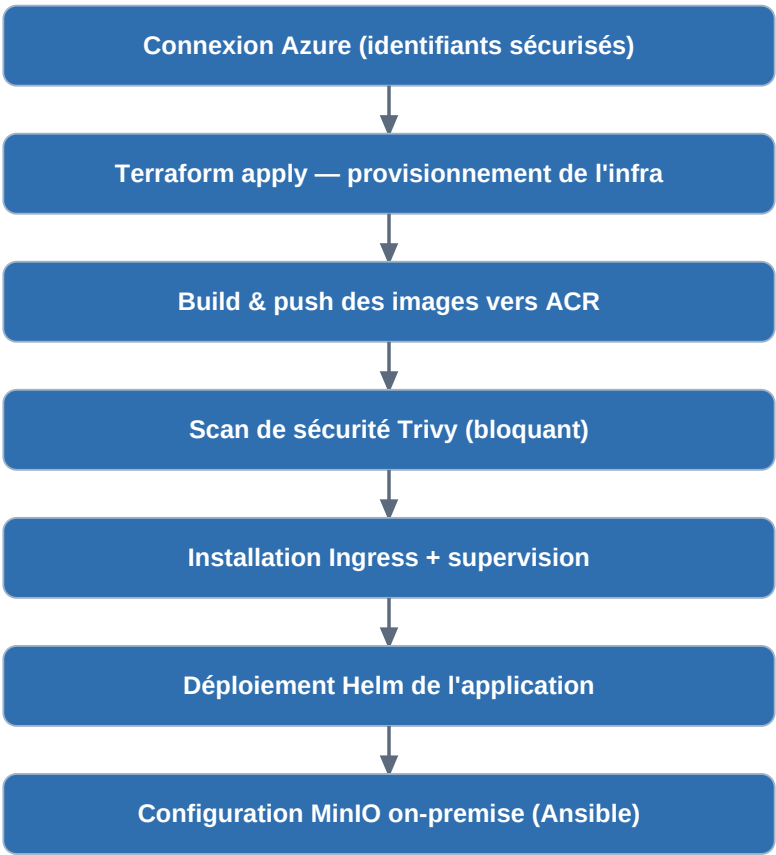
Membre	Contributions concrètes au projet
Zaafir Mougammadou Zaccaria	Conception des modules Terraform (groupe de ressources, AKS, ACR, PostgreSQL, réseau, budget), développement de l'API FastAPI et du frontend, conteneurisation et déploiement applicatif Helm.
Elyess Rjafellah	Conception des workflows GitHub Actions (intégration, sécurité, déploiement, sauvegarde planifiée), automatisation du provisionnement, état distant et mécanismes d'auto-réparation du pipeline.
Adame Nianghane	Mise en place de la supervision (Prometheus, Grafana, Loki, Alertmanager) et des tableaux de bord, intégration des scans de sécurité dans la CI, durcissement (RBAC, TLS, secrets, conteneurs non-root).
Anis Douadi	Packaging Helm de l'application, autoscaling et politiques réseau, provisionnement de la VM on-premise, configuration de MinIO chiffré et des playbooks Ansible de sauvegarde et de restauration.

5. Présentation de la solution technique

La solution repose sur un ensemble cohérent de technologies, chacune choisie pour sa pertinence et son coût d'exploitation :

Domaine	Technologies retenues
Application	Python 3.11, FastAPI, SQLAlchemy, Pydantic ; React 18 + Vite
Données	PostgreSQL 16 (Azure Flexible Server, TLS obligatoire)
Conteneurs / Orchestration	Docker (images multi-stage non-root), Kubernetes (AKS), Helm
Infra & configuration	Terraform (provider azurearm), Ansible
CI/CD	GitHub Actions (déploiement, sauvegarde, intégration, sécurité)
Observabilité	Prometheus, Grafana, Loki, Alertmanager
Sécurité	Trivy, CodeQL, Gitleaks, RBAC, NetworkPolicies, TLS, secrets chiffrés
On-premise / PRA	VM Linux, MinIO chiffré (SSE), sauvegarde et restauration Ansible

Déploiement automatisé. Un simple envoi de code sur la branche principale, ou un déclenchement manuel, suffit à provisionner l'infrastructure, construire et analyser les images, puis déployer l'application — sans aucune manipulation manuelle. Le pipeline enchaîne la connexion à Azure, l'application Terraform, la construction et la publication des images, un scan de vulnérabilités bloquant, l'installation de l'Ingress et de la supervision, le déploiement de l'application puis la configuration du stockage de sauvegarde. Il est conçu pour se rétablir seul en cas d'interruption.



Orchestration et résilience. L'application est packagée sous forme de chart Helm et exécutée sur Kubernetes, qui assure le redémarrage automatique des conteneurs défaillants et l'ajustement du

nombre de répliques selon la charge. Les conteneurs s'exécutent sans privilèges, le réseau interne est cloisonné et l'accès à l'API du cluster est régi par des rôles (RBAC).

Observabilité et sécurité. L'API expose ses propres métriques, collectées par Prometheus et visualisées dans Grafana ; les journaux sont centralisés via Loki et les alertes routées vers une messagerie d'équipe. À chaque livraison, le code et les images sont analysés par Trivy, CodeQL et Gitleaks, et aucune image présentant une vulnérabilité critique n'est déployée.

Reprise d'activité. Les données sont sauvegardées de façon chiffrée vers le stockage MinIO hébergé sur le site on-premise, et restaurables automatiquement via Ansible. Ce mécanisme a été **validé en conditions réelles** : après avoir créé une donnée témoin, l'avoir sauvegardée puis volontairement supprimée, la restauration a permis de la retrouver à l'identique. L'ensemble de la solution a été déployé, testé de bout en bout et jugé conforme aux attentes du cahier des charges.



Cycle de reprise d'activité validé en conditions réelles : de la création d'une donnée à sa récupération après sauvegarde et perte simulée.